

TP1 - Apache2 / Nginx

+ Attention la plupart des commande en mode administrateur (sudo)

Création de MVs :

RT BOX (Bridge / interne)

Deux mv Ubuntu Desktop

Mise à jour (sudo apt update)

Teste de connectivité(ping)

Installer le serveur apache 2 (apt install apache 2)

1. Installer les différents paquetages et configurer le service apache2 du serveur web

1.1 Installer les paquets

- Faire une installation de base du serveur apache2 en utilisant la [documentation officielle](#)
- Vérifier que votre service est fonctionnel.
- Insérer dans votre compte-rendu tous les éléments qui permettent de s'assurer de la bonne configuration de votre service (méthodes, extrait de fichier de configuration, extrait des commandes exécutées avec leur



résultat, ...).

1.2. Configuration de base d'apache2

\$ cd /etc/apache2

```
administrateur@rt-mv:/etc/apache2$ ls -alh
total 96K
drwxr-xr-x  8 root root 4,0K févr. 24 13:49 .
drwxr-xr-x 131 root root 12K févr. 24 13:58 ..
-rw-r--r--  1 root root 7,1K juil. 17 2024 apache2.conf
drwxr-xr-x  2 root root 4,0K févr. 24 13:49 conf-available
drwxr-xr-x  2 root root 4,0K févr. 24 13:49 conf-enabled
-rw-r--r--  1 root root 1,8K déc.  4 2023 envvars
-rw-r--r--  1 root root 31K déc.  4 2023 magic
drwxr-xr-x  2 root root 12K févr. 24 13:49 mods-available
drwxr-xr-x  2 root root 4,0K févr. 24 13:49 mods-enabled
-rw-r--r--  1 root root 320 déc.  4 2023 ports.conf
drwxr-xr-x  2 root root 4,0K févr. 24 13:49 sites-available
drwxr-xr-x  2 root root 4,0K févr. 24 13:49 sites-enabled
administrateur@rt-mv:/etc/apache2$
```

Modules

Il est possible d'ajouter des modules à Apache afin d'étendre les fonctionnalités du serveur web.

Les modules disponibles sont répertoriés ici : [/etc/apache2/mods-available](#)

mod_php

[PHP](#) est un langage de programmation qui permet principalement de générer du contenu HTML. C'est donc une solution très largement utilisée pour créer des applications web ou des [sites internet dynamiques](#).

Il est très couramment utilisé en conjonction d'Apache. Voir [LAMP](#).

Le module *mod_php* fournit un interpréteur (ou plus exactement une [SAPI](#)) PHP intégré à Apache. C'est la méthode la plus simple, mais aussi la moins performante, pour utiliser PHP avec Apache.

Une autre méthode consiste à utiliser [PHP-FPM](#). C'est une solution plus performante, plus souple et plus sécurisée, mais plus complexe à mettre en œuvre. Nous ne traiterons pas de ce cas ici.

Pour installer et activer *mod_php* sous ubuntu, on utilise cette commande :

```
sudo apt install libapache2-mod-php
```

Mais encore une fois, mieux vaut vous reporter à la documentation de [LAMP](#) à ce sujet.

mod_rewrite

[mod_rewrite](#) permet de réécrire des [URL](#).

Il s'agit généralement de remplacer le chemin, le nom de la page, et la chaîne de requête de l'[URL](#) par une chaîne de caractère en concordance avec la structure et le contenu du site.

De nombreux services ont recours à cette pratique afin d'augmenter leur référencement et de clarifier leur contenu.

Ces services utilisent parfois une terminologie exotique pour mentionner cette pratique : [WordPress](#) parle par ex. de *permalinks*.

On peut aussi s'en servir pour déployer une [API](#) web propre.
On utilise aussi parfois cette méthode afin de mettre en place des [redirections web](#) un peu complexes.

Pour activer ce module on utilisera la commande

```
sudo a2enmod rewrite
```

L'utilisation de *mod_rewrite* est relativement complexe et fait appel à des [expressions régulières](#). N'hésitez pas à consulter [la documentation officielle](#) à ce sujet.

Pour la rédaction des expressions régulières, vous pouvez vous aider de services en ligne tels que [regex101.com](#) ou [regexr.com](#)

Un petit exemple :

RewriteEngine on

```
RewriteRule ^page-([0-9]+)$ /page.php?id=$1 [L]
```

Lorsque l'on tape dans la barre d'adresse `www.example.com/page-*` c'est la page `www.example.com/page.php?id=*` qui s'affiche

`^` et `$` dans le bloc modèle signifient respectivement le début et la fin d'une ligne.

`$1` est une variable qui récupère la valeur entre les premières parenthèses du bloc modèle

`[L]=[last]` signifie que si cette règle s'applique on n'en cherche pas d'autre

On peut écrire ces règles de réécriture dans une section [<VirtualHost>](#) ou [<Directory>](#) (le comportement n'est pas le même), ou dans un fichier [.htaccess](#).

N'oubliez pas de modifier le virtualhost en conséquence sinon les règles de réécriture `.htaccess` ne fonctionneront pas correctement. Voici un exemple de configuration qui marche :

```
<Directory /var/www/html>  
    AllowOverride All  
</Directory>
```

mod_proxy

Comme [son nom l'indique](#), [mod_proxy](#) permet à apache de relayer des requêtes depuis ou vers un service tiers.

Ce module peut par exemple être utile lorsqu'on utilise d'autres serveurs HTTP en plus d'Apache sur un même serveur web, afin d'éviter d'avoir à accéder au contenu web sur d'autres ports que le port 80 ou 443.

L'activation de ce module sur un serveur ouvert sur le web est dangereuse. Une mauvaise configuration permet par exemple à un intrus d'accéder aux services disponibles sur le réseau local, ou d'usurper votre adresse IP. N'activez ce module que si vous êtes certain de ce que vous faites.

Voir [la documentation officielle à ce sujet](#).

Pour activer ce module :

```
sudo a2enmod proxy proxy_http proxy_wstunnel
sudo systemctl reload apache2
```

Voici un exemple de configuration qui permet d'accéder à un serveur [emby](http://example.com/emby) à l'adresse <http://example.com/emby> plutôt que <http://example.com:8096> :

```
<Location /emby>
    ProxyPass http://localhost:8096/
    ProxyPassReverse http://localhost:8096/
    Require all granted
    SetEnv proxy-nokeepalive 1
    SetEnv proxy-sendchunked 1
</Location>
```

mod_userdir

Il peut être utile, et c'est prévu par Apache, que chaque utilisateur puisse mettre un contenu web à disposition depuis son espace personnel (dans le répertoire [home](#)).

Ce contenu sera accessible à l'adresse http://example.com/~nom_de_lutilisateur, ou dans la plupart des cas : http://localhost/~nom_de_lutilisateur.

Pour mettre cette configuration en place, on crée un répertoire [public_html](#) dans son espace personnel et on lui donne les droits de lecture et d'exécution :

```
mkdir ~/public_html
echo 'Mon site personnel' > ~/public_html/index.html
chmod -R 755 ~/public_html
```

On active ensuite le module *mod_userdir* :

```
sudo a2enmod userdir
sudo systemctl reload apache2
```

Le message *Mon site personnel* devrait alors être lisible à l'adresse http://localhost/~nom_de_lutilisateur

1.3. Pages protégées :

Création de répertoire :

```
cd var/www/
mkdir auth
Création des fichiers :
dans : cd var/www/auth/
touch users
touch group
```

on se trouve dans /var/www/html

Création de répertoire :

mkdir securise

touch .htaccess

nano .htaccess

dans .htaccess :

AuthType Basic

AuthName "Réservé aux enseignants"

AuthUserFile /var/www/auth/users

AuthGroupFile /var/www/auth/group

Require group enseign

Création des utilisateurs et du groupe

- htpasswd -c /var/www/auth/users merchez

- echo "enseign : adam

"> /var/www/auth/group

Activer le module authz_groupfile :

- a2enmod authz_groupfile

Dans cd /etc/apache2/

nano apache2.conf :

<Directory /var/www/html/securise>

AllowOverride All

</Directory>

Notes : redémarrer le serveur apache2 avec la commande : systemctl reload apache2

1.4. Limitation de la bande passante :

Activer le module ratelimit : a2enmod ratelimit

Dans le fichiers htaccess :

nano .htaccess :

SetOutputFilter RATE_LIMIT

SetEnv rate-limit 40

1.5. Pages personnelles :

On active ensuite le module *mod_userdir* :

```
sudo a2enmod userdir
sudo systemctl reload apache2
Puis :
adduser adam_tmimi
mkdir /home/adam_tmimi/public_html
touch nano
nano index.html
chmod 755 adam_tmimi
chmod 755 public_html
```

1.6. Serveurs virtuels :

crée deux fichier

```
/etc/apache2/mods-available# nano /etc/apache2/sites-available/client.conf
/etc/apache2/mods-available# sudo nano /etc/apache2/sites-available/vendeur.conf
```

copier le contenu de :

```
sudo nano /etc/apache2/sites-available/000-default.conf
```

y ajouter :

Vendeur

```
<VirtualHost *:80>
    ServerName vendeur.localhost
    DocumentRoot /var/www/vendeur
    ErrorLog ${APACHE_LOG_DIR}/vendeur_error.log
    CustomLog ${APACHE_LOG_DIR}/vendeur_access.log combined
</VirtualHost>
```

Client :

```
<VirtualHost *:80>
    ServerName client.localhost
    DocumentRoot /var/www/client
    ErrorLog ${APACHE_LOG_DIR}/client_error.log
    CustomLog ${APACHE_LOG_DIR}/client_access.log combined
</VirtualHost>
```

Puis :

```
sudo mkdir /var/www/vendeur
sudo mkdir /var/www/client
```

puis :
sudo a2ensite vendeur.conf
sudo a2ensite client.conf

Machine LEMP

<https://www.digitalocean.com/community/tutorials/how-to-install-linux-nginx-mysql-php-lemp-stack-on-ubuntu>

sudo apt update
sudo apt install nginx
limit_conn_zone \$binary_remote_addr zone=addr:10m;

sudo ufw app list

Output

Available applications:

Nginx Full

Nginx HTTP

Nginx HTTPS

sudo ufw allow 'Nginx HTTP'

sudo ufw status

Output

Status: active

To	Action	From
--	-----	----
OpenSSH	ALLOW	Anywhere
Nginx HTTP	ALLOW	Anywhere
OpenSSH (v6)	ALLOW	Anywhere (v6)
Nginx HTTP (v6)	ALLOW	Anywhere (v6)

ip addr show

http://server_domain_or_IP

Step 2 — Installing MySQL

sudo apt install mysql-server

sudo mysql_secure_installation

```
sudo mysql
exit
```

Step 3 – Installing PHP

```
sudo apt install php8.1-fpm php-mysql
```

Step 4 — Configuring Nginx to Use the PHP Processor

```
sudo mkdir /var/www/your_domain
sudo chown -R $USER:$USER /var/www/your_domain
sudo nano /etc/nginx/sites-available/your_domain
```

```
server {
    listen 80;
    server_name your_domain www.your_domain;
    root /var/www/your_domain;

    index index.html index.htm index.php;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~ /\.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/var/run/php/php8.1-fpm.sock;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/your_domain /etc/nginx/sites-enabled/
sudo unlink /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
nano /var/www/your_domain/index.html
```



```
<html>
<head>
  <title>your_domain website</title>
</head>
<body>
  <h1>Hello World!</h1>

  <p>This is the landing page of <strong>your_domain</strong>.</p>
</body>
</html>
```

```
http://server_domain_or_IP
```

Step 5 –Testing PHP with Nginx

```
nano /var/www/your_domain/info.php
```

```
<?php
phpinfo();
```

```
http://server_domain_or_IP/info.php
```

```
sudo rm /var/www/your_domain/info.php
```

<https://www.digitalocean.com/community/tutorials/how-to-install-and-secure-phpmyadmin-with-nginx-on-an-ubuntu-20-04-server>

Step 1 — Installing phpMyAdmin

```
sudo apt update
sudo apt install phpmyadmin
```

Note: Assuming you installed MySQL by following [Step 2 of the prerequisite LEMP stack tutorial](#), you may have decided to enable the Validate Password plugin. As of this writing, enabling this component will trigger an error when the phpMyAdmin installation process attempts to set a password for the **phpmyadmin** user:

```
Configuring phpmyadmin
An error occurred while installing the database:

mysql said: ERROR 1819 (HY000) at line 1: Your password does not satisfy the
current policy requirements . Your options are:
* abort - Causes the operation to fail; you will need to downgrade,
reinstall, reconfigure this package, or otherwise manually intervene
to continue using it. This will usually also impact your ability to
install other packages until the installation failure is resolved.
* retry - Prompts once more with all the configuration questions
(including ones you may have missed due to the debconf priority
setting) and makes another attempt at performing the operation.
* retry (skip questions) - Immediately attempts the operation again,
skipping all questions. This is normally useful only if you have
solved the underlying problem since the time the error occurred.
* ignore - Continues the operation ignoring dbconfig-common errors.
This will usually leave this package without a functional database.

Next step for database installation:

abort
retry
retry (skip questions)
ignore

<Ok>
```

```
sudo mysql
mysql -u root -p
UNINSTALL COMPONENT "file://component_validate_password";
```

```
exit
INSTALL COMPONENT "file://component_validate_password";
```

```
sudo ln -s /usr/share/phpmyadmin /var/www/your_domain/phpmyadmin
```

```
http://server_domain_or_IP/phpmyadmin
```